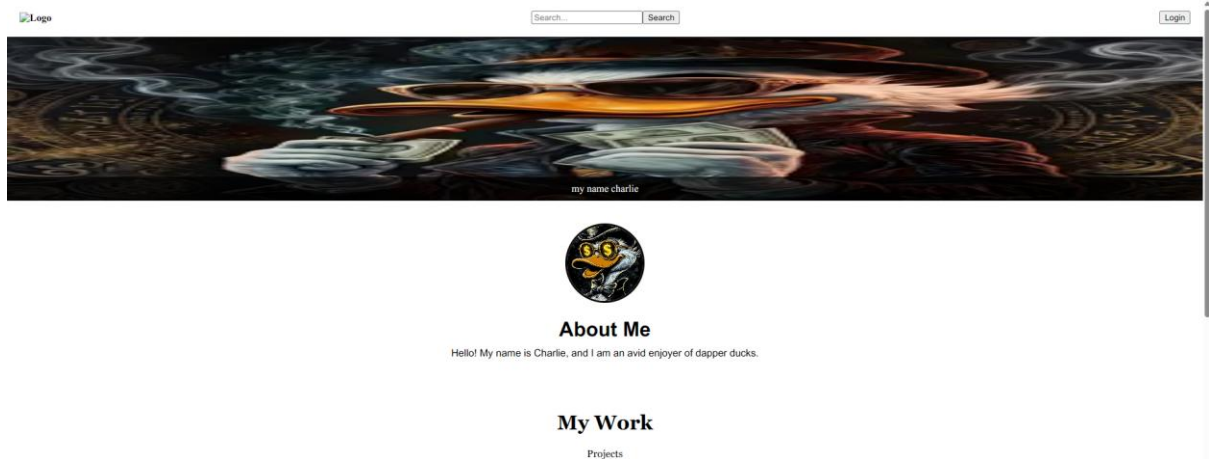


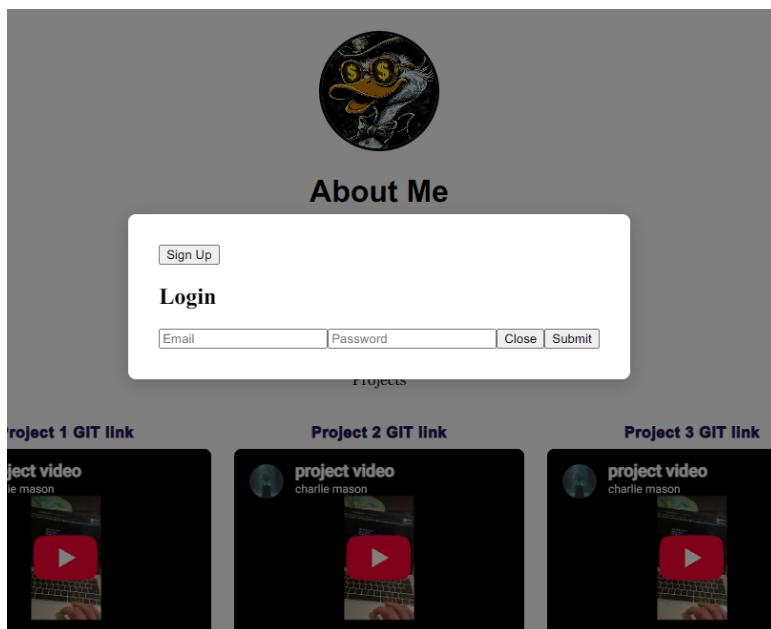
Task 4p: Login and Registration Page

Header bar:



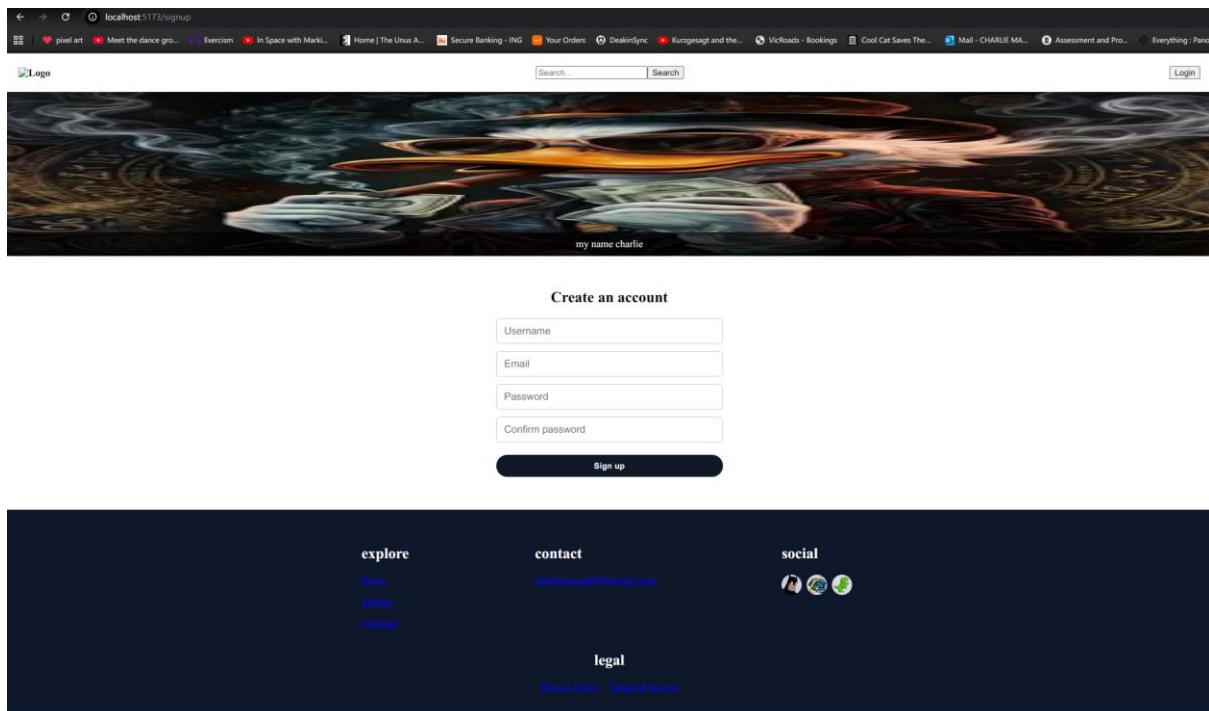
This header contains the login button and the return logo button

login overlay:



pressing the login button creates this overlay that disables the background and lets you choose to login or create a new account if you don't have one.

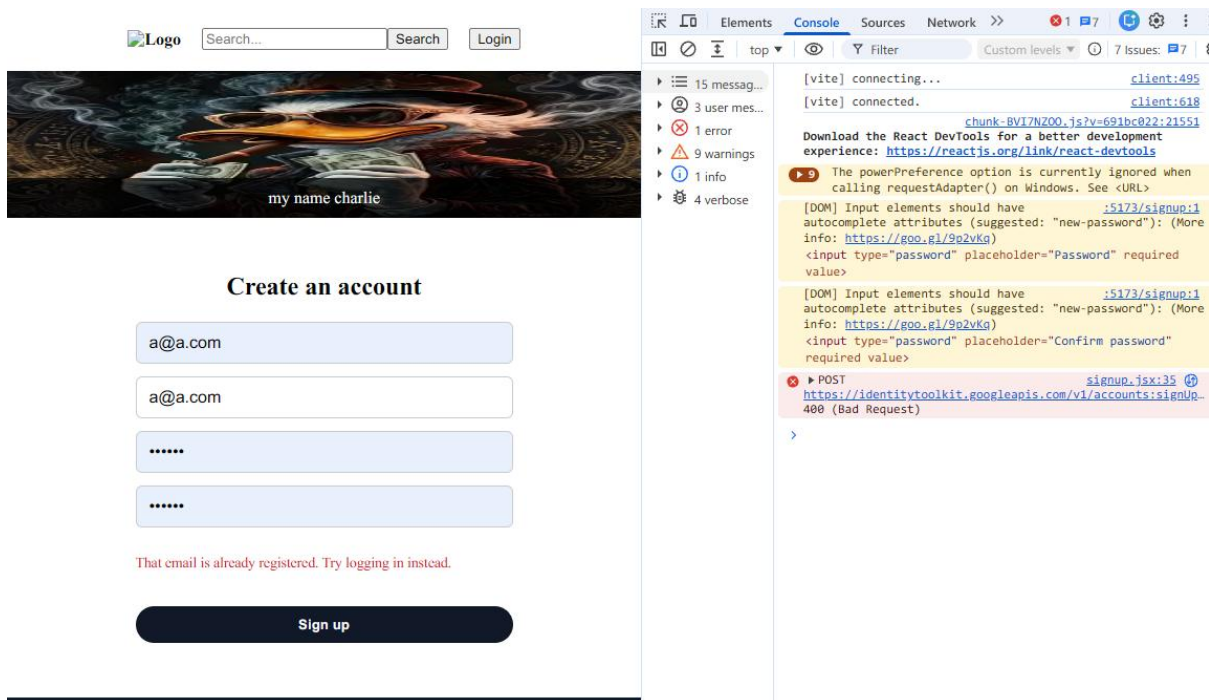
Signup page:



The screenshot shows a web browser at localhost:5173/signup. The page has a dark header with a logo, a search bar, and a login button. Below the header is a large, abstract, colorful image with the text "my name charlie" at the bottom. The main content area is titled "Create an account" and contains four input fields: "Username", "Email", "Password", and "Confirm password". A "Sign up" button is at the bottom of the form. Below the form is a footer with three columns: "explore" (with links to Home, About, Contact), "contact" (with a link to contact@charlie.com), and "social" (with icons for GitHub, Twitter, and Facebook). A "legal" section with links to Privacy Policy and Terms of Service is also present.

Hitting the signup page redirects you to a new signup page where you can create a new account.

Duplicate accounts:



The screenshot shows the same signup page as before, but with the "Email" field filled with "a@a.com". Below the "Confirm password" field, a red error message reads: "That email is already registered. Try logging in instead." The "Sign up" button is still visible. To the right of the page is a Chrome DevTools console showing a POST request to https://identitytoolkit.googleapis.com/v1/accounts:signUp-400 (Bad Request). The console also shows several warnings and messages related to the application's state and development tools.

If you attempt to create a new account with an email that already exists then firebase will reject your request and a note will appear to inform the user

Invalid Login states:

The image displays four screenshots of a login form, illustrating different states of the application:

- Initial State:** The form is empty, with the title "Login" and a "Sign Up" button. The input fields are labeled "Email" and "Password". Below the fields, it says "Please enter both email and password." and there are "Close" and "Submit" buttons.
- Filled State:** The "Email" field contains "w@qa" and the "Password" field contains "Password". The "Please enter both email and password." message is still present.
- Error State:** The "Email" field contains "w". A red error message box appears, stating: "Please include an '@' in the email address. 'w' is missing an '@'." The "Please enter both email and password." message is still present.
- Success State:** The "Email" field contains "w@qa" and the "Password" field contains "Password". The message "Incorrect email or password. Please try again." is displayed below the fields.

The html box for the email is set to type email to validate email input. The handle submit function validates all the inputs are correct and actually entered.

```
<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
  placeholder="Email"
  autoComplete="username"
/>

if (!email || !password) {
  setError('Please enter both email and password.');
```

```
  return;
}

setLoading(true);
try {
  await signInWithEmailAndPassword(auth, email, password);
  setIsLoggedIn(true);
  setShowLogin(false);
  navigate('/');
} catch (err) {
  // Firebase gives specific error codes (auth/invalid-credential, au
  setError('Incorrect email or password. Please try again.');
```

```
  finally {
    setLoading(false);
  }
}
};
```

Signup code:

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');

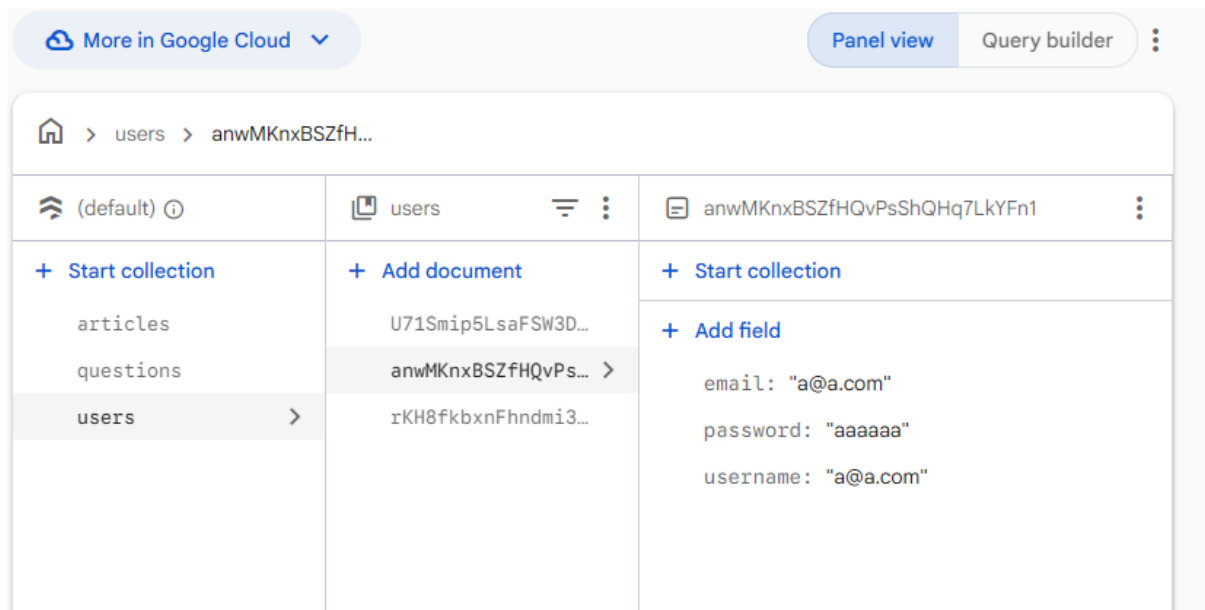
  if (password !== confirmPassword) {
    setError('Passwords do not match. ');
    return;
  }

  setSubmitting(true);
  try {
    const userCredential = await createUserWithEmailAndPassword(
      auth,
      email,
      password
    );
    const user = userCredential.user;
    await setDoc(doc(db, 'users', user.uid), {
      username,
      email,
      password,
    });

    navigate('/'); // redirect after successful signup
  } catch (err) {
    setError(mapFirebaseError(err));
  } finally {
    setSubmitting(false);
  }
};
```

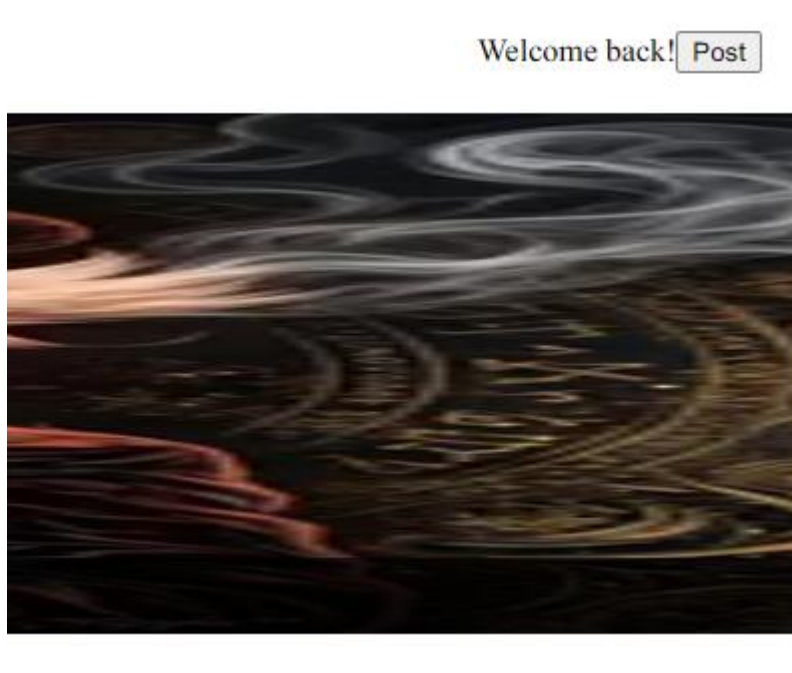
Apon hitting submit the signup.jsx code runs the function that confirms the password validating and then attempts to create a new set of user credentials using the firebase library built in user with email and password features and catches any errors like the duplicate accounts we had earlier.

Firestore:



All the accounts are stored in firestore like so.

Login:



Apon successful login the page changes an reveals the post button and remove the login button

Login.jsx:

```
function Login({ onClose, isLoggedIn, setIsLoggedIn }) {
  const [showLogin, setShowLogin] = useState(false);
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleLoginClick = () => setShowLogin((prev) => !prev);

  const handleClose = () => {
    setShowLogin(false);
    if (onClose) onClose();
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    if (!email || !password) {
      setError('Please enter both email and password.');
      return;
    }

    setLoading(true);
    try {
      await signInWithEmailAndPassword(auth, email, password);
      setIsLoggedIn(true);
      setShowLogin(false);
      navigate('/');
    } catch (err) {
      // Firebase gives specific error codes (auth/invalid-credential, auth/too-many-requests, etc.)
      setError('Incorrect email or password. Please try again.');
```

The logged in state controls whether we can see the login or post button and is changed by validating the submitted username and password which is handled by the firebase library.

Page navigation:

```
function App() {
  return (
    <BrowserRouter>
      <Header />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/signup" element={<Signup />} />
      </Routes>
      <Footer />
    </BrowserRouter>
  );
}
```

These are the routes that all the buttons can go to wrapped in the header and footer elements.

Video link: <https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=1230ba37-3779-4439-9f0e-b4b4000613e6>

GITLAB link: https://gitlab.deakin.edu.au/s223391013/SIT_313_frontend_reimport/-/tree/e846a28dce6f8d15c764980a62f4b96778345664/p4